

ЛЦТ 2026 — Vehicle Re-ID:

техническая документация

Команда AdequacyOFF

29 сентября 2026

Обзор решения

Сервис идентификации автомобиля по фото с городской камеры видеонаблюдения без опоры на госномер, который часто грязный, закрыт или не попадает в кадр. Пользователь (оператор) загружает фото машины — сервис возвращает цифровой профиль (цвет, кузов) и ранжированный список вероятных совпадений из базы кадров с других камер, с процентом схожести.

Сценарии применения: розыск угнанной машины, реконструкция маршрута по камерам, поиск скрывшегося с места ДТП.

Архитектура и пайплайн

Кадр с камеры → YOLO (детектор) → кроп машины
→ DINOv2-Large + ArcFace (эмбеддер) → 1024-мерный вектор
→ FAISS (векторный поиск, cosine similarity) → топ-K похожих
→ CLIP zero-shot (атрибуты: цвет, кузов, видимые особенности)
→ цифровой профиль + ранжированный список совпадений

Компоненты:

- **Детекция** — YOLOv8 (Ultralytics), предобученный на COCO, без дообучения — для выделения машины в кадре этого достаточно.
- **Эмбеддер** — facebook/dinov2-large (1024-мерный CLS-токен), дообученный ArcFace-головой на собственном объединённом датасете.
- **Векторный поиск** — FAISS IndexFlatIP поверх L2-нормированных векторов (эквивалент косинусной близости).
- **Атрибуты** — zero-shot CLIP (ViT-L-14) для цвета, типа кузова и видимых особенностей (тонировка, наклейки, повреждения, диски).
- **Backend** — FastAPI (POST /identify, POST /enroll, GET /health).
- **Frontend** — React + TypeScript (Vite), статический сайт, ходит на backend по сети.

Почему DINOv2-Large, а не CLIP

Сравнили DINOv2 и CLIP как базовую модель для эмбединга. DINOv2 лучше держит мелкие визуальные детали — диски, потёртости, наклейки — то, что реально отличает одну машину от другой той же модели и цвета; CLIP оптимизирован под семантическое сопоставление с текстом, а не под fine-grained визуальное различие.

Backbone дообучен целиком (не LoRA) — на 10 387 identities важнее ёмкость модели, чем экономия видеопамати на A100 80GB.

Как решалась проблема ракурса и освещения

Городские камеры дают низкое разрешение, ночную съёмку, случайный ракурс и сжатие — совсем не то, что студийные фото объявлений (основной источник обучающих данных). Для переноса на реальные условия:

1. `scripts/camera_augment.py` — на лету имитирует камеру поверх студийных фото: blur, JPEG-сжатие, шум, имитация ночи, лёгкая перспектива сверху (применяется с вероятностью 0.4 в датасет-лоадере).
2. Часть обучающих данных (BoxCars116k) — реальные снимки со 137 стационарных камер видеонаблюдения, а не студийные фото — прямой сигнал устойчивости к ракурсу/освещению без всякой аугментации.

Датасет

Источник	Что даёт	Объём
Drom.ru (свой парсинг)	Студийные фото объявлений, несколько ракурсов на объявление, метки марка/кузов/цвет	97 моделей, 8066 фото
BoxCars116k	Реальные камеры видеонаблюдения, треки одной машины	8001 трек, 29 176 фото
Итого	Объединённый датасет для ArcFace	10 387 identities

Identity = одна физическая машина (папка `ad_id` у Drom или `track_id` у BoxCars). Датасеты объединяются на уровне классов ArcFace — без физического слияния файлов.

Датасеты, доступ к которым не получен за время хакатона (форма запроса у авторов, ответ не пришёл): VeRi-776, VehicleID, VeRi-Wild, CityFlow-ReID. Из-за этого нет сравнения с внешним отраслевым бенчмарком — все метрики ниже посчитаны на собственном честном val-сплите.

Обучение эмбеддера: путь к результату

Train/val сплит — **по identity**, не по фото (90/10, fixed seed=42): если резать по фото, один ракурс машины может попасть в train, другой — в val, и val перестает быть проверкой на *невиданных* машинах. Val-identity полностью исключены из классов ArcFace — это принципиально: классификация по классам, которые видели на train, не измеряет re-id способность.

Прогон	Backbone	Batch	Epochs	margin	mAP	Rank-1	Rank-5
Baseline (без обучения)	dinov2-base	—	0	—	0.297	0.352	0.514
v3	dinov2-base	64	15	0.30	0.243 (хуже baseline)	0.238	0.447
v4	dinov2-base	64	15	0.05	0.508	0.534	0.765
v5 (финал)	dinov2-large	128	25	0.05	0.759	0.793	0.915

Честная история находки: первый прогон (margin=0.30 — типовое дефолтное значение ArcFace) не сходилась вообще — train accuracy оставалась нулевой все 15 эпох, а итоговый mAP получился хуже нетронутого претрейна. Причина — margin=0.30 слишком агрессивен для холодного старта с 10 000+ классов: логиты верного класса ещё не разделены, а margin делает их только хуже. Снижение margin до 0.05 и переход на dinov2-large с батчем 128 дали honest val mAP 0.759 — рост в 3.1 раза относительно сломанного прогона.

Гиперпараметры финального прогона: facebook/dinov2-large, ArcFace (scale=30, margin=0.05), AdamW (backbone lr=3e-5, head lr=3e-4, отдельные группы), cosine schedule, batch=128, 25 эпох, camera_prob=0.4.

Честный анализ ошибок

На val-сплите (1038 query, честно отложенные identity): **215 из 1038 (20.7%) неверных топ-1 ответов** — согласуется с Rank-1=0.793.

Наблюдения по характеру ошибок:

- **Похожие модели путает чаще, чем разные.** Пример из теста: query — чёрный Peugeot 408 (peugeot_408/299 122 239/00.jpg). Топ-1 — **другой** чёрный Peugeot 408 того же цвета и комплектации (peugeot_408/366 380 529/02.jpg, score 0.93). Правильный ответ (другое фото того же query-объявления) — на 2-м месте, score 0.87. Модель верно выучила “это Peugeot 408 чёрного цвета”, но не различает две конкретные машины этой модели по мелким деталям при высоком визуальном сходстве.
- **Ошибка редко фатальна.** Даже при неверном топ-1 правильный ответ почти всегда попадает в топ-3–4 — не теряется совсем. Для сценария “оператор смотрит ранжированный список”, а не “автоматическое решение”, это не критично.
- **Реальные камеры (BoxCars) сложнее, чем студийные фото Drom.** Ниже разрешение, случайный ракурс и худшее освещение увеличивают долю ошибок по сравнению с чистыми студийными фото — ожидаемо и совпадает с целью аугментаций.

API

FastAPI, запускается `uvicorn src.api.main:app --host 0.0.0.0 --port 8800` на сервере с GPU (все ML-модели грузятся туда).

Метод	Путь	Описание
GET	/health	{"status":"ok","index_size":N}
POST	/identify	multipart/form-data, поле file — фото. Возвращает профиль + ранжированный список совпадений
POST	/enroll	multipart/form-data, поле file + query-параметр camera_id. Добавляет кадр в векторный индекс
GET	/data/...	Статика — кропы для превью совпадений

CORS открыт (`allow_origins=["*"]`) — сайт и API намеренно разнесены на разные origin.

Как развернуть

Два независимых способа. Первый — полностью автономный, у себя на GPU. Второй — быстрый просмотр через наш уже поднятый сервер (для демонстрации вживую, требует пароль, который передаётся отдельно).

Способ 1 — полностью у себя (рекомендуется для проверки)

Нужна машина с GPU (CUDA) — на CPU тоже запустится, но инференс будет медленным. Обученный чекпоинт (1.2 ГБ) и демо-индекс не лежат в git (слишком тяжёлые для обычного репозитория) — они в [GitHub Release](#) этого репозитория.

```
git clone https://github.com/GeDeKo/lct2026-vehicle-reid.git
cd lct2026-vehicle-reid
```

```
python3 -m venv .venv && source .venv/bin/activate
# сначала torch под CUDA (или cpu-сборка) — иначе requirements.txt ниже
# подтянет torch с обычного PyPI и поставит не тот билд
pip install torch torchvision --index-url https://download.pytorch.org/whl/cu121
pip install -r requirements.txt
```

```
./scripts/download_release_assets.sh # чекпоинт + демо-индекс из Release
```

```
uvicorn src.api.main:app --host 0.0.0.0 --port 8800 # бэкэнд
```

В отдельном терминале — фронтенд:

```
cd web
npm install
npm run build && npx vite preview --port 5500
```

Открыть `http://localhost:5500` — по умолчанию сайт уже настроен ходить на `http://localhost:8800` (см. `web/.env.example`; переопределяется через `VITE_API_BASE` при сборке или `?api=http://host:port` в адресе страницы). `GET /health` должен ответить `{"status":"ok","index_size":57}`.

Способ 2 — быстрый просмотр через наш сервер

Инференс уже запущен нами на A100 — для просмотра без разворачивания своей GPU достаточно поднять только веб-клиент и пробросить порт:

```
JUMP_PASS='<передан отдельно>' ./web/keep_tunnel_alive.sh &
curl http://localhost:8800/health # ждём {"status":"ok",...}
```

```
cd web && npm install && npm run build && npx vite preview --port 5500
```

Пароль от прокси-сервера передаётся организаторам отдельно, вне репозитория. Скрипт сам переподнимает туннель, если тот оборвётся. Подробности и план на случай сбоя — в `web/README.md`.

Известные ограничения и план развития

- **Марка/модель в профиле пока не подключены в прод.** Головы атрибутов уже обучены (`scripts/train_attributes.py`, `brand_acc` 0.70, `body_acc` 0.81, `color_acc` 0.78 на своём `val`), но не `wire`-нуты в `AttributeTagger` — сейчас там только `zero-shot CLIP` для цвета/кузова.
- **Реранкинг (`k-reciprocal`) не реализован** — поиск сейчас чисто по косинусной близости, без переранжирования по обратным соседям.
- **Фильтр по времени/месту камер не реализован** — нет геоданных камер в модели данных (`VehicleRecord` хранит только `camera_id` и `timestamp`).
- **Нет сравнения с внешним бенчмарком** (`VeRi-776` и т.п.) — доступ не получен за время хакатона, все метрики — на собственном `val`-сплите.
- **Масштабирование:** `FAISS IndexFlatIP` — предел прототипа; при 500k+ записей переход на `IndexIVFFlat/HNSW` (интерфейс класса не меняется), плюс асинхронная очередь энкодинга кадров вместо синхронной обработки по одному запросу.